



UCSC

University of Colombo, Sri Lanka

University of Colombo School of Computing

BACHELOR OF SCIENCE IN COMPUTER SCIENCE

First Year Examination — Semester II– UCSC AY19 [held in March/April/May 2023]

SCS1209 — Object Oriented Programming

(Two (2) Hours)

Answer ALL questions

240

Number of Pages = 16

Number of Questions = 4

To be completed by the candidate

Index Number

--	--	--	--	--	--	--	--	--	--

Important Instructions to candidates:

- Students should answer in the medium of English language only using the space provided in this question paper.
- Note that questions appear on both sides of the paper. If a page is not printed, please inform the supervisor immediately.
- Write your index number **CLEARLY** on each and every page of this question paper.
- The duration of the paper is **Two (2)** hours.
- This paper has **4** questions in **16** pages (including the Cover Page).
- Answer **all** the questions.
- Calculators and any electronic device capable of storing and retrieving text including electronic dictionaries, smart watches and mobile phones are not allowed.
- Do not tear off any part of this question paper. Under no circumstances may this paper, used or unused, be removed from the Examination Hall by a candidate.

To be completed by the examiners

1	
2	
3	
4	
Total	

Index Number

--	--	--	--	--	--	--	--

1. (a). Determine whether the following statements are true or false. Place write True next to true statements. If a statement is False, rewrite it to correct the statement in the space provided below.

[5 marks]

i. Procedural programming is better than object-oriented programming (OOP) in terms of code organization and readability, making it the preferred choice for complex software projects.	
ii. The operation of the copy constructor must be always explicitly defined by the user.	
iii. Static functions in object-oriented programming (OOP) can be accessed without creating an instance of the class, and hence a non-member function can call a static member function using the syntax " <i>ClassName::staticMemberFunctionName()</i> ".	
iv. A friend function can access a class's private data without being a member of the class.	
v. In object-oriented programming, every instance of a class shares the same set of member variables and is not independent of other instances of the same class.	

Correct incorrect statements here.

Index Number

--	--	--	--	--	--	--	--

- (b). Consider the following program written in C++ and answer the questions from (i) to (v).

```
#include <iostream>
using namespace std;

class Engine {
private:
int cylinders; // Number of cylinders
double liters; // Capacity in liters

public:
Engine(int c = 4, double l = 1.5);
~Engine();
void displayEngine();
} e;
```

_____ (1)

```
{
cout<<"Constructor Engine init the variables"<<endl;
cout<<"Cylinders: "<<cylinders<<endl;
cout<<"Liters: "<<liters<<endl; }
```

```
void Engine::displayEngine() {
cout<<"Engine Number of cylinders: "<<cylinders<<endl;
cout<<"Engine Capacity in liters: "<<liters<<endl; }
```

_____ (2)

```
{
cout<<"Destructor Called - Engine:"<<cylinders<<"(C)"<<endl; }
```

```
class Car {
private:
int modelNumber; // Model number
Engine motor; // Car's engine
public:
Car(int m, int c, double l);
};
```

_____ (3)

```
{
cout<<"Constructor Car init the variables"<<endl;
cout<<"Model No: "<<modelNumber<<endl; }
```

```
int main() {
Car c(2,6,2.8); ..... (4)
}
```

Index Number

--	--	--	--	--	--	--	--

- i. Complete the code denoted in (1) outside the Engine class to create a constructor that matches the given constructor signature using a constructor initialization list.

[3 marks]

--

- ii. Complete the code denoted in (2) to define the destructor.

[2 marks]

--

- iii. Complete the code denoted in (3) to define the constructor for Car Class to match the given signature using a constructor initialization list.

[3 marks]

--

- iv. Suppose you have written all the missing code segments syntactically correct. Write down the console output of the above program.

[4 marks]

--

Index Number

--	--	--	--	--	--	--	--

- v. Assuming the following two code segments are added into the main after the code segment denoted by (4), requiring a constructor to be added to the Car class, provide the relevant code segment to fulfill this new requirement.

```
Engine myEngine(8,2.4);
```

```
Car myCar(3,myEngine);
```

Note: The constructor body should include the following.

```
cout<<"Constructor Car finished  
initializing the variables"<<endl;
```

```
cout<<"Model No: "<<modelNumber<<endl;
```

[4 marks]

--

- vi. After adding the code segment mentioned in b (v), write down the console output. Assume that the constructor is written syntactically correct.

[4 marks]

--

Index Number

--	--	--	--	--	--	--	--

2. In object-oriented programming (OOP), operator overloading allows developers to redefine the behavior of operators for user-defined data types, enhancing code flexibility and expressiveness.

- (a). Compare and contrast three (3) differences between an Operator function and a Normal Function.

[5 marks]

--

- (b). Consider the following code segment and answer the questions from (i) to (iii).

```
class MyClass {  
    private:  
        int num;  
  
    public:  
        MyClass () {  
----- (A)  
        cout << "Constructor 1 Invoked\n";  
        MyClass(int x) {  
----- (B)  
            num = x;  
            cout << "Constructor 2 Invoked\n";  
  
            MyClass(MyClass& y) {  
----- (C)  
                num = y.num;  
                cout << "Constructor 3 Invoked\n";  
  
                void display() {cout << num;}  
  
                void operator = (MyClass& a) {  
                    num = a.num++;  
                    cout << "Operator Function Invoked\n"; }  
};
```

Index Number

--	--	--	--	--	--	--	--

```
int main() {  
    MyClass obj1(22);  
    MyClass obj2;  
    obj2 = obj1;  
    cout << "\nOBJ2="; obj2.display();  
    cout<<endl;  
    MyClass obj3(obj1);  
    cout << "\nOBJ3="; obj3.display();  
    cout<<endl;  
}
```

- i. Specify the special name given to the constructors named from A to C.

[3 marks]

--

- ii. Write down the console output of the above program.

[8 marks]

--

--	--	--	--	--	--	--	--

- [5 marks]**

This image shows a single sheet of white paper with horizontal blue or grey ruling lines. The lines are evenly spaced and run across the width of the page. There is no handwriting or printed text on the paper.

- | | |
|--|--|
| i. double x = double(3) / 2; is an example for an implicit data conversion. | |
| ii. To convert from a user-defined class to a basic type, you would most likely use an overloaded = operator. | |
| iii. To convert from a basic type to a user-defined class, you would most likely use a built-in conversion operator. | |
| iv. int to char is a basic data type conversion. | |

8

Index Number

--	--	--	--	--	--	--	--

3. (a). **Only ONE** answer is correct in the following 15 MCQs.

Cross or color the **best suite answer** among the given options in the table below.

[2 x 15 = 30 marks]

i.	(A)	(B)	(C)	(D)	(E)
iii.	(A)	(B)	(C)	(D)	(E)
v.	(A)	(B)	(C)	(D)	(E)
vii.	(A)	(B)	(C)	(D)	(E)
ix.	(A)	(B)	(C)	(D)	(E)
xi.	(A)	(B)	(C)	(D)	(E)
xiii.	(A)	(B)	(C)	(D)	(E)
xv.	(A)	(B)	(C)	(D)	(E)

ii.	(A)	(B)	(C)	(D)	(E)
iv.	(A)	(B)	(C)	(D)	(E)
vi.	(A)	(B)	(C)	(D)	(E)
viii.	(A)	(B)	(C)	(D)	(E)
x.	(A)	(B)	(C)	(D)	(E)
xii.	(A)	(B)	(C)	(D)	(E)
xiv.	(A)	(B)	(C)	(D)	(E)

i. Consider the following three statements regarding the Inheritance in OOP.

- I. It is a mechanism that allows code reuse.
- II. It is a mechanism that allows to use meaningful names.
- III. Mammal and Bat has the relationship define as Inheritance.

Which of the above statement(s) is/are TRUE?

- | | | |
|--------------|---------------------|--------------------|
| A. I only. | B. II and III only. | C. I and III only. |
| D. III only. | E. I, II, and III. | |

ii. Which of the following is TRUE according to the following class definition?

```
class A : public B, private C {
    // body of the class
}
```

- | |
|--|
| <ul style="list-style-type: none"> A. All public, protected and private data in B will become public to A. B. Both public and protected data in B will become public to A. C. Both public and protected data in C will become private to A. D. only public data in B will beocme public to C. E. A cannot access any data in C. |
|--|

Index Number

--	--	--	--	--	--	--	--

iii. What is the output of the following program written in C++?

```
class A {
    public:
        A() {cout<<"A";};
}

class B {
    public:
        B() {cout<<"B";}
        B(int x) {cout<<x;};
}

class C: public B, public A {
    public:
        C(int x) {cout<<x;};
}

int main() {
    C(25);
}
```

A. AB25
D. BA25

B. A2525
E. 25BA

C. B2525

iv. What is the output of the following program written in C++?

```
class A {
    public: int a=20;};

class B: public A {
    public: int b;};

class C: public A {
    public: int c;};

class D: public B, public C {
    public: int d=15;};

int main() {
    D d;
    d.a = 10;
    cout<<d.a;
}
```

A. 20

B. 15

C. 10

D. Compilation Error

E. Run time Error

Index Number

--	--	--	--	--	--	--	--

- v. Consider the following three statements regarding the Virtual Functions in C++.

- I. It enables Inheritance in C++.
- II. It is mandatory to override all the virtual functions.
- III. There is always an additional performance penalty when using virtual functions.

Which of the above statement(s) is/are TRUE?

- | | | |
|--------------|---------------------|--------------------|
| A. I only. | B. II and III only. | C. I and III only. |
| D. III only. | E. I, II, and III. | |

- vi. What is the output of the following program written in C++?

```
class Shape {
public:
    void area() {cout<<"void";}
    int area() {cout<<"int";}
    char area() {cout<<"char";}
};

int main() {
    Shape s;
    s.area();
}
```

- | | | |
|----------------------|-------------------|---------|
| A. void | B. int | C. char |
| D. Compilation Error | E. Run time Error | |

- vii. Consider the following three statements regarding the Function Overriding in OOP.

- I. It is the way to achieve the compile time polymorphism.
- II. It is mandatory to use the same name to override a class.
- III. You can change the parameter list when you override a function.

Which of the above statement(s) is/are TRUE?

- | | | |
|---------------------|--------------------|-------------------|
| A. I only. | B. II only. | C. I and II only. |
| D. II and III only. | E. I, II, and III. | |

Index Number

--	--	--	--	--	--	--	--

- viii. Consider the following three statements regarding the function definition given below.

```
virtual int func(int x, int y=0) = 0;
```

- I. This function returns 0 when it calls.
- II. You can call this function with only one integer parameter.
- III. You cannot create objects from the class to which this function belongs.

Which of the above statement(s) is/are TRUE?

- | | | |
|--------------|--------------------|---------------------|
| A. I only. | B. II only. | C. II and III only. |
| D. III only. | E. I, II, and III. | |

- ix. What is the primary purpose of Encapsulation in OOP?

- | |
|--|
| A. To facilitate code reuse at the implementation. |
| B. To ensure data integrity and security. |
| C. To enhance code readability. |
| D. To facilitate the data abstraction. |
| E. To enable polymorphism and inheritance. |

- x. Consider the following three statements regarding the templates in C++.

- I. It enables dynamic memory allocation.
- II. It supports generic programming.
- III. It enhances the code portability across different platforms.

Which of the above statement(s) is/are TRUE?

- | | | |
|--------------------|--------------------|---------------------|
| A. I only. | B. II only. | C. II and III only. |
| D. I and III only. | E. I, II, and III. | |

- xi. Consider the following three statements regarding the logical errors.

- I. They occur because of incorrect syntax of the code.
- II. They typically lead to runtime errors.
- III. They do not harm for the outcome of the program.

Which of the above statement(s) is/are TRUE?

- | | | |
|---------------------|--------------------|-------------------|
| A. I only. | B. II only. | C. I and II only. |
| D. II and III only. | E. I, II, and III. | |

Index Number

--	--	--	--	--	--	--	--

xii. What is the output of the following C++ program?

```
template <typename T, typename U, typename V>
T abc(T x, U y, V z) {
    return y * z; }

int main() {
    cout << abc<char, double, int>('A', 33.0, 2) << ", ";
    cout << abc<int, int, double>(7.5, 3, 2.5) << endl;
}
```

- | | | |
|-----------|----------------------|---------|
| A. A, 7.5 | B. A, 7 | C. B, 7 |
| D. B, 7.5 | E. Compilation Error | |

xiii. Consider the following definition of a template.

```
template <typename T, int n>
T func(T x) { return x + n; }
```

Which of the following is NOT a correct call of the function func() ?

- | |
|-------------------------|
| A. func<int, 3>(5) |
| B. func<char, 'A'>('B') |
| C. func<double, 2.5>(3) |
| D. func<char, 5>('A') |
| E. func<double, 5>(3)) |

xiv. What type of errors can handle through exception handling?

- | | | |
|---------------------|------------------------|-------------------|
| A. Resource errors. | B. Syntax errors. | C. Logical errors |
| D. Run time errors | E. Compilation errors. | |

xv. What is the output of the following C++ program?

```
int main() {
    cout << "K";
    try {throw 'L'; cout << "L"; }
    catch(int x) {cout << "M"; }
    catch(...) {cout << "N"; }
    cout << "O";}
```

- | | | |
|--------|----------|---------|
| A. KMO | B. LNO | C. KLNO |
| D. KNO | E. KLMNO | |

Index Number

--	--	--	--	--	--	--	--

4. (a). Write the output of the following piece of programs written in C++. Assume that all the other required lines of codes are in place to run programs.

```
i. class A {
    public:
        A() {cout<<"A";}
        A(int x) {cout<<"xA";}};

class B: public A {
    public:
        B(int x) {cout<<"B";}};

int main() {
    B b(20); }
```

[2 mark]

```
.....

ii. class A {
    public:
        A() {cout<<"A";}
        A(int x) {cout<<"xA";}};

class B {
    public:
        B() {cout<<"B";}
        B(int x) {cout<<"xB";}};

class C: public B, private A{
    public:
        C(int x) {cout<<"C";}};

int main() {
    C c(20); }
```

[3 marks]

```
.....

iii. class A {
    public:
        A() {cout<<"A";}
        B() {cout<<"B";}
        B(int x) {cout<<"xB";}};

int main() {
    A a; a.B(20); a.B();}
```

[2 mark]

Index Number

--	--	--	--	--	--	--	--

iv. `template <typename T, typename U, int n=0>
U abc(T x, U y) { return x + y + n; }`

```
int main() {  
    cout << abc<int, char>('A', 1);  
    cout << abc<char, int, 10>('A', 1) << endl;  
}
```

[3 mark]

.....

v. `class A {
 public:
 A() {cout<<"A"; }
 ~A() {cout<<" ~A"; } };`

```
int main( ) {  
  
    try {  
        A a;  
        throw 10;  
    }  
  
    catch (int x) {cout<<x; }  
}
```

[2 mark]

.....

vi. `class A {
 public:
 A() {cout<<"A";} };`

```
class B: public A {  
    public:  
        B() {cout<<"B";} };
```

```
int main( ) {  
    try { B b; throw b; }  
    catch (A a) {cout<<"A"; }  
    catch (B b) {cout<<"B"; }  
    catch (...) {cout<<"C"; }  
}
```

[3 mark]

.....

Index Number

--	--	--	--	--	--	--	--

- (b). Provide sample C++ code that demonstrates the implementation of OOP concept named Encapsulation using private attributes.

[5 marks]
